

Árboles de decisión. Funciones de costo y optimización.

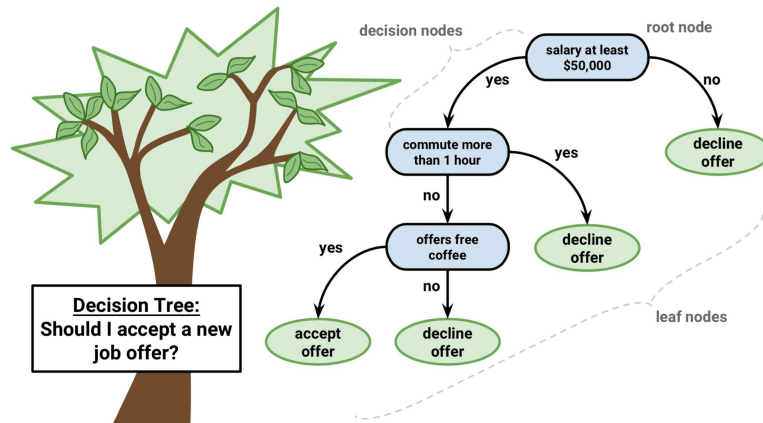
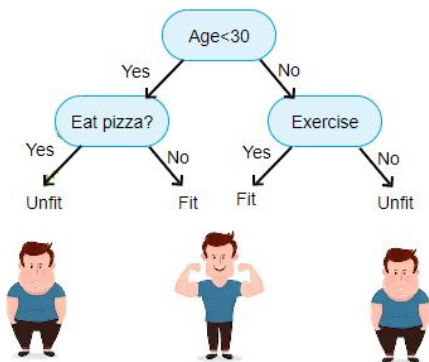


Árboles de decisión

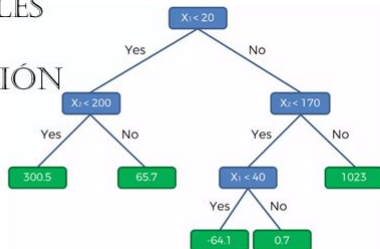


Árboles de decisión

- Un árbol de decisión es una forma gráfica y analítica de analizar y describir un conjunto de datos, con el objetivo de predecir una respuesta de interés en función de diversas variables de entrada



ÁRBOLES DE REGRESIÓN

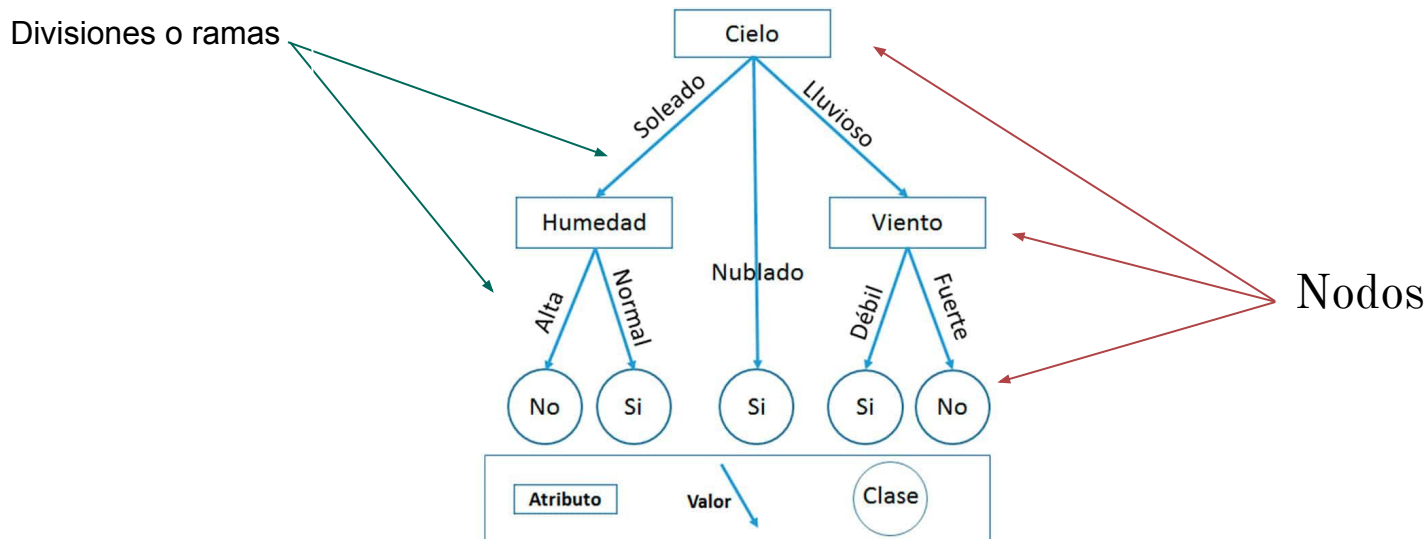


Este tipo de modelos tiene una amplia variedad de aplicaciones, entre otros:

- Predicción. Crear reglas y utilizarlas para predecir eventos futuros, como la probabilidad de entrar en mora en un crédito o el posible valor de reventa de un vehículo o una casa.
- Segmentación. Identificar a las personas que son susceptibles de ser miembros de un grupo en particular.
- Selección de variables. Identificar un subconjunto útil de predictores de un gran conjunto de variables para ajustar algún modelo paramétrico.
- Identificación de interacciones. Identificar las relaciones que corresponden únicamente a los subgrupos específicos y prever éstas en un modelo paramétrico.
- Recodificación de variables. Agrupar categorías y/o discretizar variables continuas con mínima pérdida de información.

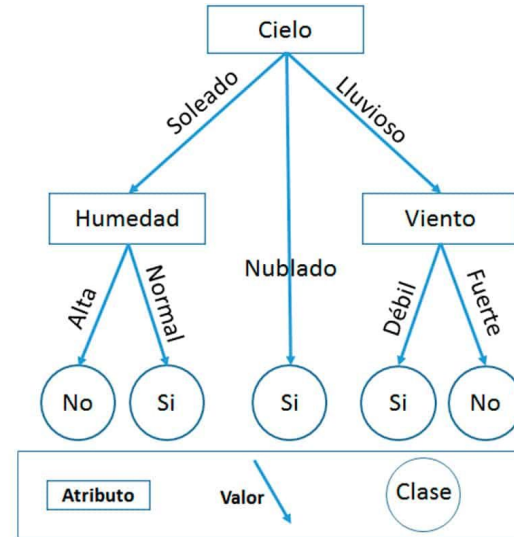
Árboles de decisión

- La apariencia de un árbol es a de un árbol invertido con el tronco o la raíz hacia arriba y ramas que crecen hacia abajo
- Las ramas se generan subdividiendo los datos, buscando generar en cada partición, subgrupos de datos más homogéneos con respecto de la variable respuesta



Árboles de decisión

La estructura final del árbol queda constituida por una red de preguntas (simples) cuyas respuestas indican que rama del árbol seguir, hasta arribar a un nodo terminal, que representa la **decisión final** para la observación o registro analizado



Las particiones que se generan en cada instancia, crean **grupos mutuamente excluyentes y exhaustivos**, de modo que cada una de las observaciones será enviada a un único nodo terminal

Complejidad y aprendizaje

- Aprender el árbol de decisión más simple (más chico) es un problema computacionalmente difícil ya que el espacio de búsqueda de todos los árboles posibles crece exponencialmente con la cantidad de características o variables.
- Se debe recurrir a heurísticas voraces (greedy)
 - Comenzar con un árbol vacío
 - Generar una partición usando **siguiente mejor atributo**
 - Paso anterior de forma recursiva

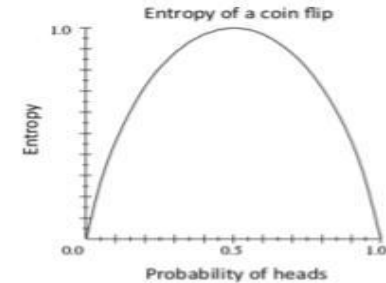
Medida de incertidumbre

- Cómo seleccionar la variable de ramificación en cada partición
 - La idea es seleccionar la que “mejor separe a los datos”, donde mejor significa que genere grupos más homogéneos respecto de la variable respuesta.
- Se debe definir un **criterio de bondad de la partición**, con el cual evaluar todas las posibles particiones y así seleccionar aquella que genere los nodos menos impuros.
- Se calcula la impureza de los nodos generados y se evalúa la reducción de impureza respecto del nodo padre.
- Se elige al predictor que genere mayor reducción de impureza.

Entropía (medida de impureza)

- Para variables respuesta categórica (con k niveles), la entropía $H(Y)$ de una variable aleatoria discreta Y

$$H(Y) = - \sum_{i=1}^k P(y = y_i) \log_2 P(Y = y_i)$$



La entropía mide cuán impredecible es una variable. Si todos los valores son iguales (sin incertidumbre) la entropía es 0. Si todos los valores son igualmente probables, la entropía es máxima.

- A mayor incertidumbre, mayor entropía

Ejemplo de aplicación del algoritmo: Datos del juego...

día	estado	temperatura	humedad	viento	juego
1	nublado	alta	alta	no	si
2	nublado	media	alta	si	si
3	lluvioso	media	alta	no	si
4	lluvioso	media	alta	si	no
5	soleado	alta	alta	no	no
6	soleado	alta	alta	si	no
7	soleado	media	alta	no	no
8	nublado	baja	normal	si	si
9	nublado	alta	normal	no	si
10	lluvioso	baja	normal	no	si
11	lluvioso	media	normal	no	si
12	soleado	baja	normal	no	si
13	soleado	media	normal	si	si
14	lluvioso	baja	normal	si	no

Respuesta binaria: juego, binaria

Predictores:

- **Estado:** 3 niveles, nominal
3 particiones
- **Temperatura:** 3 niveles, ordinal
2 particiones
- **Humedad:** 2 niveles (ordinal)
1 partición
- **Viento:** 2 niveles (nominal)
1 partición

Ganancia de información

- Se calcula la entropía para el nodo padre y para los nodos hijos generados tras una partición

Reducción de Impureza

- Se hace un promedio ponderado de la entropía de los nodos hijos.
- Se compara con la entropía del nodo padre.
- El resultado indica cuánta impureza se reduce tras dividir el nodo (también llamado **ganancia de información**).

Decisión del algoritmo

- La división nunca aumenta la impureza.
- Cuanto mayor sea la reducción de impureza, mejor será la partición.

Objetivo: elegir la partición que maximice esta medida.

Partición	Total	Juego		Entropía	Disminución en la entropía	
		no	si			
	14	5	9	0,940		
lluvioso, nublado	9	2	7	0,764	0,838	0,102
soleado	5	3	2	0,971		
lluvioso, soleado	10	5	5	1,000	0,714	0,226
nublado	4	0	4	0,000		
lluvioso	5	2	3	0,971	0,937	0,003
nublado, soleado	9	3	6	0,918		

Ejemplo de aplicación del algoritmo: Datos del juego...

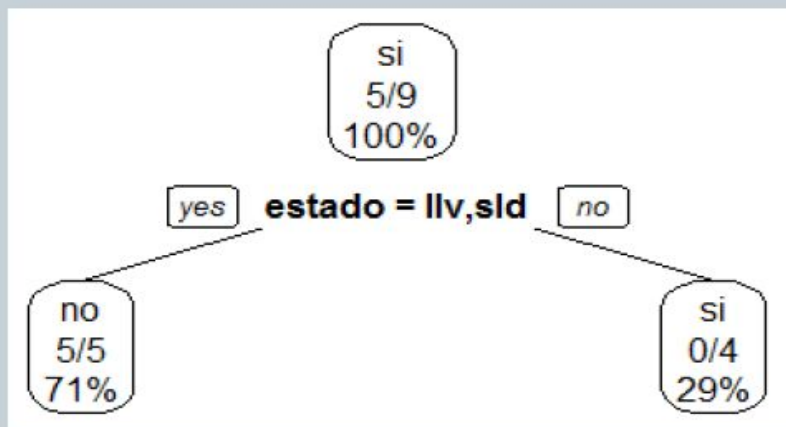
Repetimos el análisis para los restantes predictores y listamos las particiones elegidas para cada uno:

Variable		Total	Juego		Entropía		Disminución en la entropía
			no	si			
		14	5	9	0,940		
Estado	lluvioso, soleado	10	5	5	1,000	0,714	0,226
	nublado	4	0	4	0,000		
Temperatura	alta	4	2	2	1,000	0,915	0,025
	baja, media	10	3	7	0,881		
Humedad	alta	7	4	3	0,985	0,788	0,152
	normal	7	1	6	0,592		
Viento	no	8	2	6	0,811	0,892	0,048
	si	6	3	3	1,000		

El predictor que mejor separa las categorías de la respuesta es Estado.

Ejemplo de aplicación del algoritmo: Datos del juego...

Se realiza la primera partición de los datos según la variable Estado en {lluvioso, soleado} versus {nublado}:

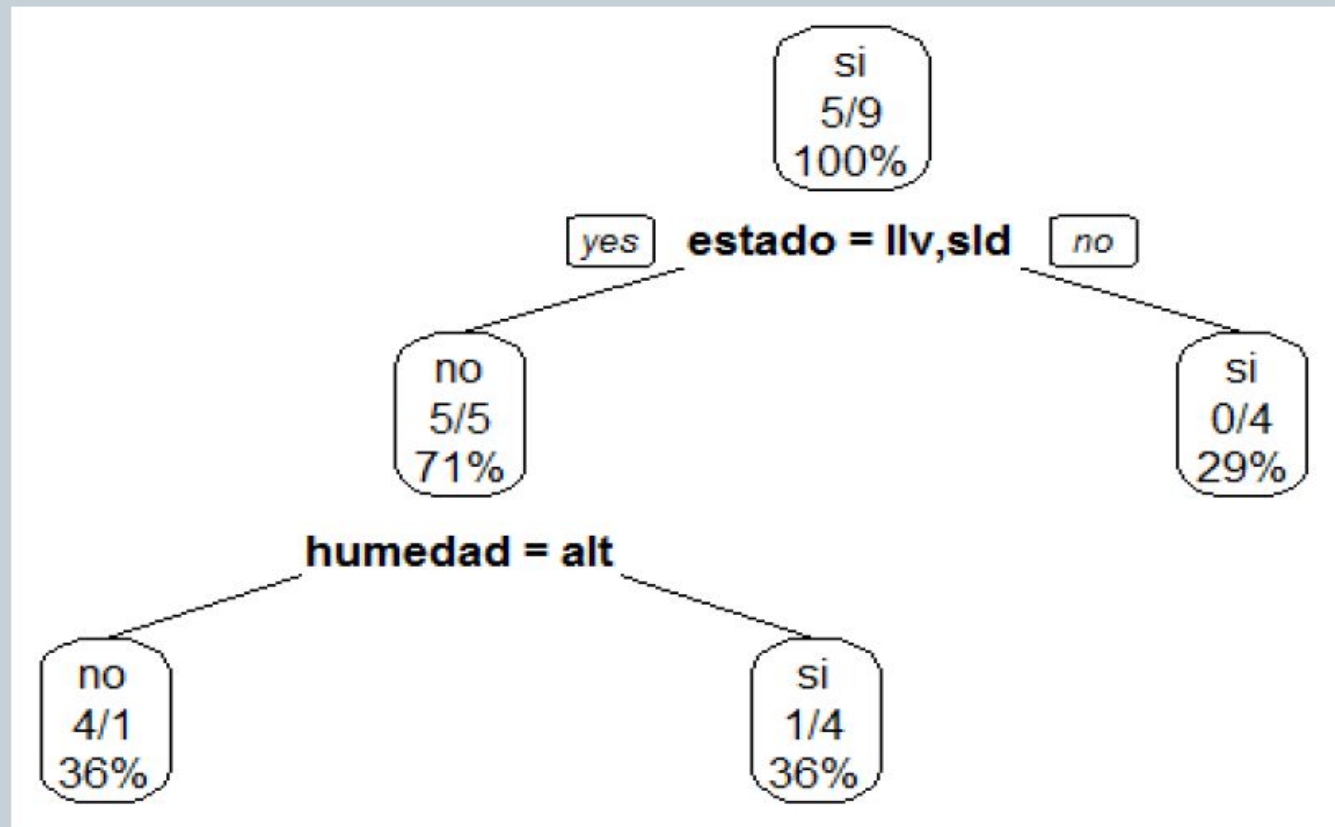


El nodo raíz contiene el 100% de los datos.
Entre ellos hay 5 registros con juego = NO y 9 registros con juego = SI.
Categoría predicha = SI.

Luego, se evalúa si los nodos hijos son nodos terminales.

Se repite el procedimiento para cada uno de los nodos no terminales.

Ejemplo de aplicación del algoritmo: Datos del juego...





¿Cuándo un nodo es terminal? – Criterios de parada

- **El nodo es puro**, es decir, los valores de la respuesta son constantes en el nodo.
- **Los predictores son constantes en el nodo** (aunque las respuestas no sean todas iguales).
- Se ha alcanzado una **profundidad máxima** para el árbol (predefinida).
- El número de observaciones en el nodo es menor que el **tamaño mínimo de nodo padre**.
- El número de observaciones en cualquiera de los nodos hijos de la mejor división del nodo, es menor que el **tamaño mínimo de nodo hijo**.
- La mejor división para el nodo no genera la **reducción de impureza mínima** deseada (predefinida).

Sobreajuste en árboles de decisión

- El error de entrenamiento es siempre cero (si no hay errores en las etiquetas)
- Poca capacidad de generalización. Son muy sensibles a los datos de entrenamiento. Se dice que son “inestables”.
- Pueden tener serios problemas para resolver algunos casos sencillos.
- Se debe inducir algún sesgo a modelos más simples
 - Fijar un límite a la profundidad del árbol
 - Número finito de muestra en cada nodo hoja
 - etc..
- Ensamble: Random Forest (bosques!) y bootstrapped trees

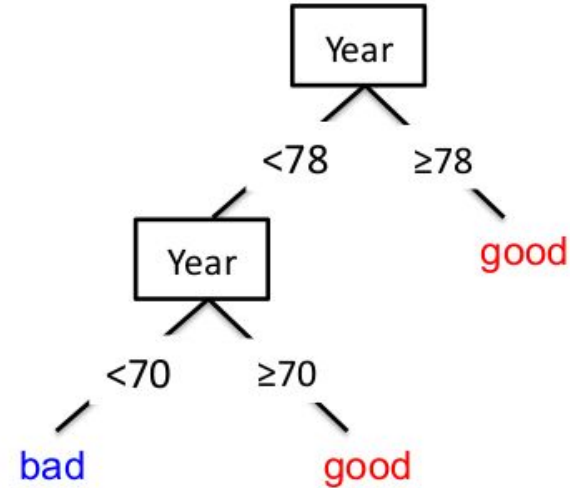
Entrada con valores reales

- Existe un número infinito de posibles particiones!

mpg	cylinders	displacemen	horsepower	weight	acceleration	modelyear	maker
good	4	97	75	2265	18.2	77	asia
bad	6	199	90	2648	15	70	america
bad	4	121	110	2600	12.8	77	europa
bad	8	350	175	4100	13	73	america
bad	6	198	95	3102	16.5	74	america
bad	4	108	94	2379	16.5	73	asia
bad	4	113	95	2228	14	71	asia
bad	8	302	139	3570	12.8	78	america
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
good	4	120	79	2625	18.6	82	america
bad	8	455	225	4425	10	70	america
good	4	107	86	2464	15.5	76	europa
bad	5	131	103	2830	15.9	78	europa

Partición mediante umbrales

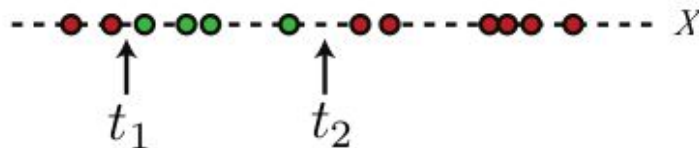
- Árboles binarios
 - partir un atributo X a un valor t
 - Una rama para $x < t$
 - Una rama para $x \geq t$
- Se deben permitir particiones de un mismo atributo en distintos niveles de un mismo camino



El conjunto de posibles umbrales

- Árboles binarios
 - partir un atributo X a un valor t
 - Una rama para $x < t$
 - Una rama para $x \geq t$
- Explorar todos los valores posibles de t es intratable
- Sólo un número finito de valores es importante
 - Ordenar X de acuerdo a los valores del atributo $\{x_1, \dots, x_N\}$
 - Considerar puntos (umbrales) entre muestras de clases distintas

$$x_i + (x_{i+1} - x_i)/2$$



- $X=[1,2,4,5,7,9]$ con etiquetas $[0,0,0,1,1,1]$, los puntos umbrales se consideran entre 4 y 5

Elegir el mejor umbral

- Supongamos una variable X y umbral t
- $IG(Y|X:t)$ denota la ganancia de información de Y para cuando particionamos X de acuerdo a t
- Definimos

$$H(Y|X:t) = p(X < t) H(Y|X < t) + p(X \geq t) H(Y|X \geq t)$$

$$IG(Y|X:t) = H(Y) - H(Y|X:t)$$

$$IG^*(Y|X) = \max_t IG(Y|X:t)$$

- Usamos $IG^*(Y|X)$ con variables continuas

Árboles de decisión. Resumen

- Uno de los modelos más utilizados en la práctica
 - Fáciles de comprender, implementar y utilizar
 - Computacionalmente eficientes
- Muchas variantes para cada selección de atributos basados en ganancia de información (ID3, C4.5, ..)
- Se pueden usar en regresión y para la estimación de densidades
- Sobreajuste por definición!
 - Heurísticas para definir árboles más simples (prunning, fixed depth, early stopping, etc)
 - ensemble de distintos árboles (eg. random forest)

Demo con notebook

05 Arboles de Decision.ipynb

—

Funciones de costo y optimización

Regresión polinomial

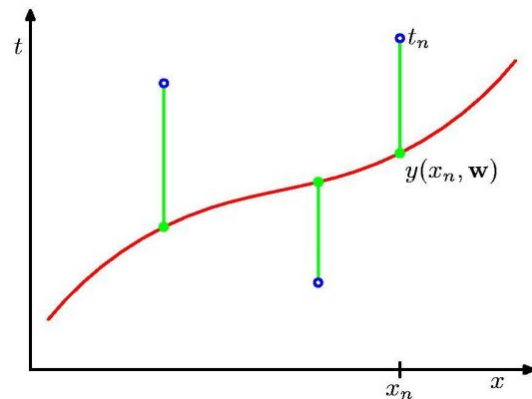
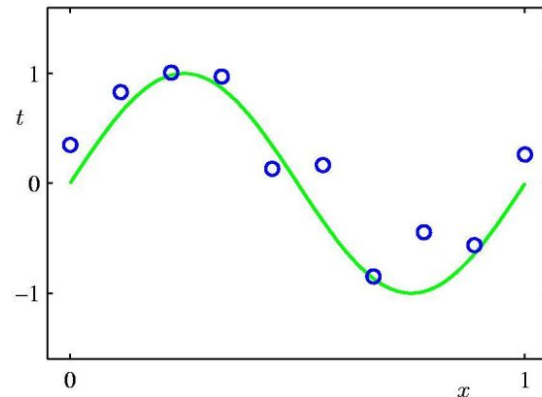
- Función de predicción lineal

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \cdots w_Mx^M$$

- Función de costo: error cuadrático
medida de error en la predicción de t mediante $y(x, \mathbf{w})$

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n, \mathbf{w}) - t_n)^2$$

- Admite una solución de forma cerrada



Regresión logística

- Datos $\{(x_1, y_1), \dots, (x_N, y_N)\}$ con $x_i \in \mathbb{R}^n, y_i \in \{0, 1\}$

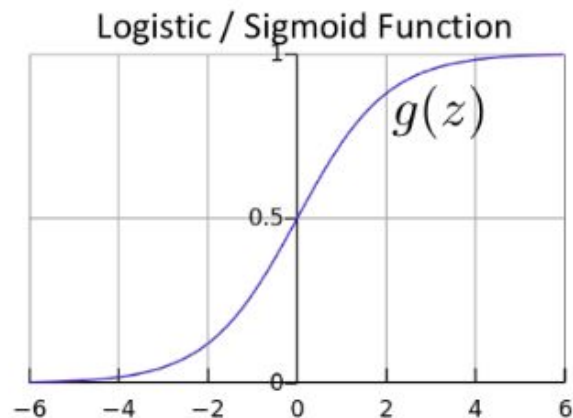
- Modelo $p(y = 1|x) = h_w(x)$

$$h_w(x) = \frac{1}{1 + \exp(-w^T x)}$$

- Función de costo:

$$L(w) = - \sum_{i=1}^N y_i \log(h_w(x_i)) + (1 - y_i) \log(1 - h_w(x_i))$$

- $h_w(x)$ no lineal $\longrightarrow$ **no admite solución cerrada**



Optimización y Aprendizaje

- Un problema típico en ML se puede escribir como

$$L(w) = \sum_{i=1}^N l(y_i, f_w(x_i)) + \lambda R(w)$$

en función de los
parámetros del modelo

Costo de predicción del
par (x,y)

regularización para controlar el sobreajuste
(overfitting). No depende de las muestras
(x,y)

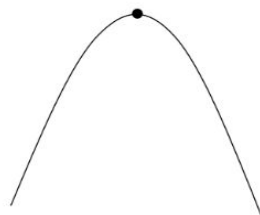
- “Aprender” significa resolver:

$$w^* = \arg \min_w L(w)$$

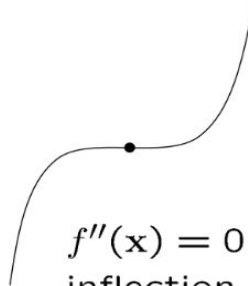
y que $f_{w^*}()$ pueda **generalizar** ejemplos no vistos

Optimización sin restricciones

- Recordemos. Caso de funciones 1D, $f : \mathbb{R} \rightarrow \mathbb{R}$
 - f tiene un **punto estacionario** en x_0 cuando $\frac{\partial f}{\partial x}(x_0) = 0$
 - La derivada segunda determina el **tipo** de punto estacionario



$f''(\mathbf{x}) \leq 0$
maximum



$f''(\mathbf{x}) = 0$
inflection



$f''(\mathbf{x}) \geq 0$
minimum

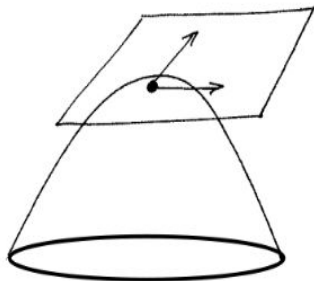
Optimización sin restricciones

- Recordemos casos de funciones nD $f : \mathbb{R}^n \rightarrow \mathbb{R}$

- f tiene un punto estacionario en x_0 cuando

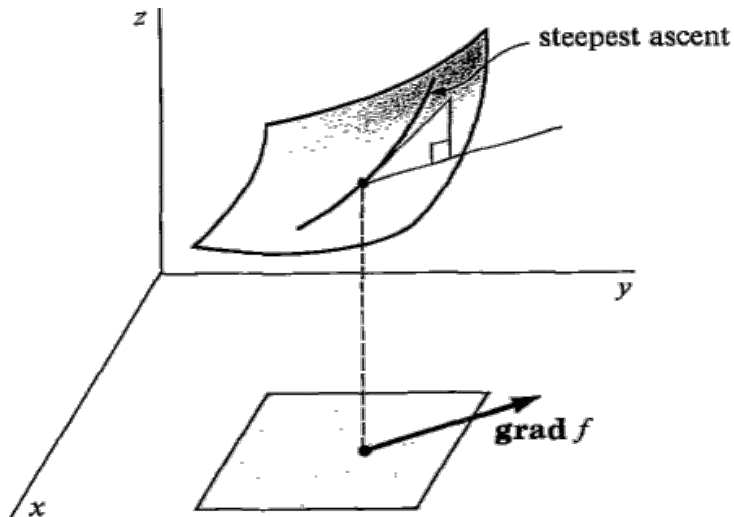
$$\nabla f(x_0) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right) (x_0) = 0$$

- el **tipo** de punto estacionario lo determina la matriz Hessiana

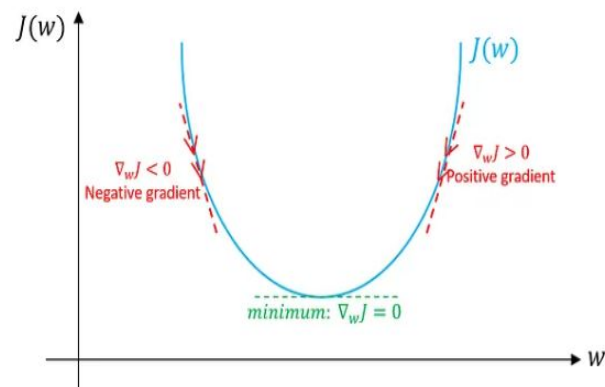


$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)^{\top} = 0$$

Optimización sin restricciones



- El gradiente en un punto da la dirección de máximo crecimiento



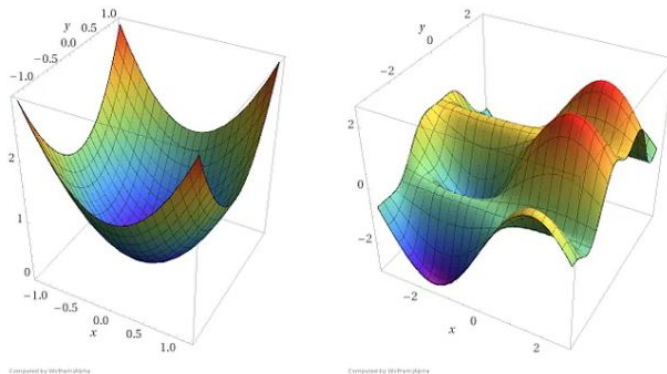
source Ref 2, Fig 1. Gradient of a function at different points

Optimización sin restricciones

- Idea general de los métodos (iterativos) de descenso: a partir de algún w , avanzar en direcciones de f decrecientes

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + \Delta \mathbf{w}^{(t)}$$

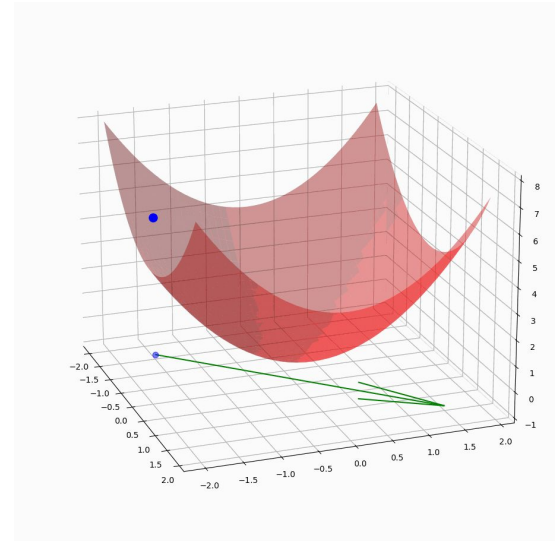
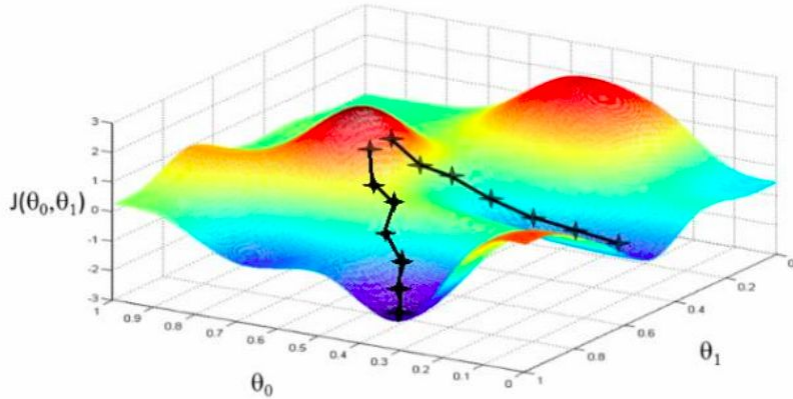
El objetivo es encontrar una dirección que nos lleve a los puntos que forman un valle en la siguiente figura



Variantes

Descenso de gradiente: Nos desplazamos hacia la dirección del valor negativo dado por el gradiente:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + \Delta \mathbf{w}^{(t)} \quad \Delta \mathbf{w}^{(t)} = -\eta_t \nabla L(\mathbf{w}^{(t)})$$



Variantes

Descenso de gradiente estocástico : se diferencia del descenso de gradiente tradicional en que actualiza los parámetros usando una sola muestra a la vez en lugar de todo el conjunto de datos.

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + \Delta \mathbf{w}^{(t)} \quad \Delta \mathbf{w}^{(t)} = -\eta_t \nabla l_n(\mathbf{w}^{(t)})$$

Newton-Rapshon (segundo orden, etc).

Descenso del gradiente

Entrada:

- Conjunto de pares de entrenamiento $D = \{(x_i, y_i)\}$
- Una tasa de aprendizaje η

Algoritmo:

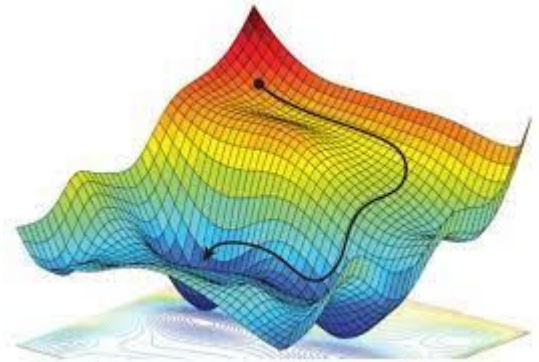
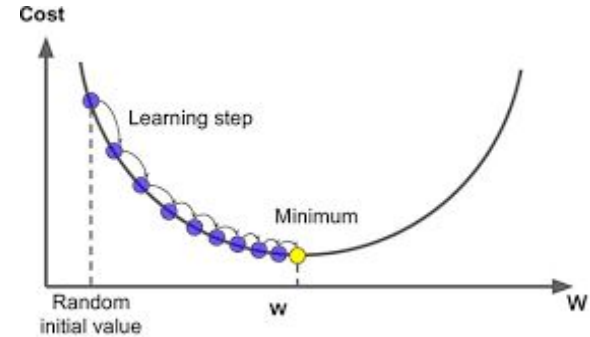
- Inicializar $w^{(0)} \in \mathbb{R}^n$
- Repetir hasta converger

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^T, D)$$

- Retornar el último $\mathbf{w}$

Convergencia

- no hay cambio (norma del gradiente menor a un epsilon)
- número de iteraciones (early stopping)



Ejemplo

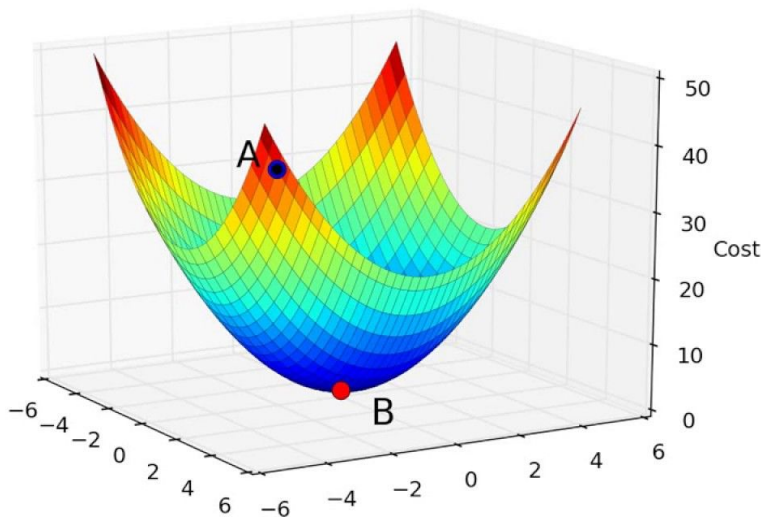
- Datos de entrenamiento $D = \{(x_i, y_i)\}, x_i \in \mathbb{R}^n, y_i \in \mathbb{R}, i = 1, \dots, N$
- **Modelo lineal** (parametrización): $f_{\mathbf{w}}(x) = \mathbf{w}^T \mathbf{x}$
- **Función de error cuadrático:** $l(f_{\mathbf{w}}(\mathbf{x}), y) = 1/2(f_{\mathbf{w}}(\mathbf{x}) - y)^2 = 1/2(\mathbf{w}^T \mathbf{x} - y)^2$
- Problema a resolver:
$$\mathbf{w}^* = \arg \min_{\mathbf{w}} L(\mathbf{w}) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N l(f_{\mathbf{w}}(\mathbf{x}_i), y_i)$$
- Paso de actualización :

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^T, D)$$

Ejemplo

- Gradiente :

$$\begin{aligned}\nabla L_{\mathbf{w}} L(\mathbf{w}, D) &= \begin{pmatrix} \frac{\partial L(\mathbf{w}, D)}{\partial w_1} \\ \vdots \\ \frac{\partial L(\mathbf{w}, D)}{\partial w_n} \end{pmatrix} = \nabla_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N l(f_{\mathbf{w}}(\mathbf{x}_i), y_i) \\ &= \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} \left[\frac{1}{2} (\mathbf{w}^T \mathbf{x}_i - y_i)^2 \right] \\ &= \frac{1}{N} \sum_{i=1}^N \underbrace{(\mathbf{w}^T \mathbf{x}_i - y_i)}_{\text{error de predicción (salida)}} \underbrace{x_i}_{\text{variable entrada}}\end{aligned}$$



Ejemplo

Entrada:

- Conjunto de pares de entrenamiento $D = \{(x_i, y_i)\}$
- Una tasa de aprendizaje η

Algoritmo:

- Inicializar $w^{(0)} \in \mathbb{R}^n$
- Repetir hasta converger

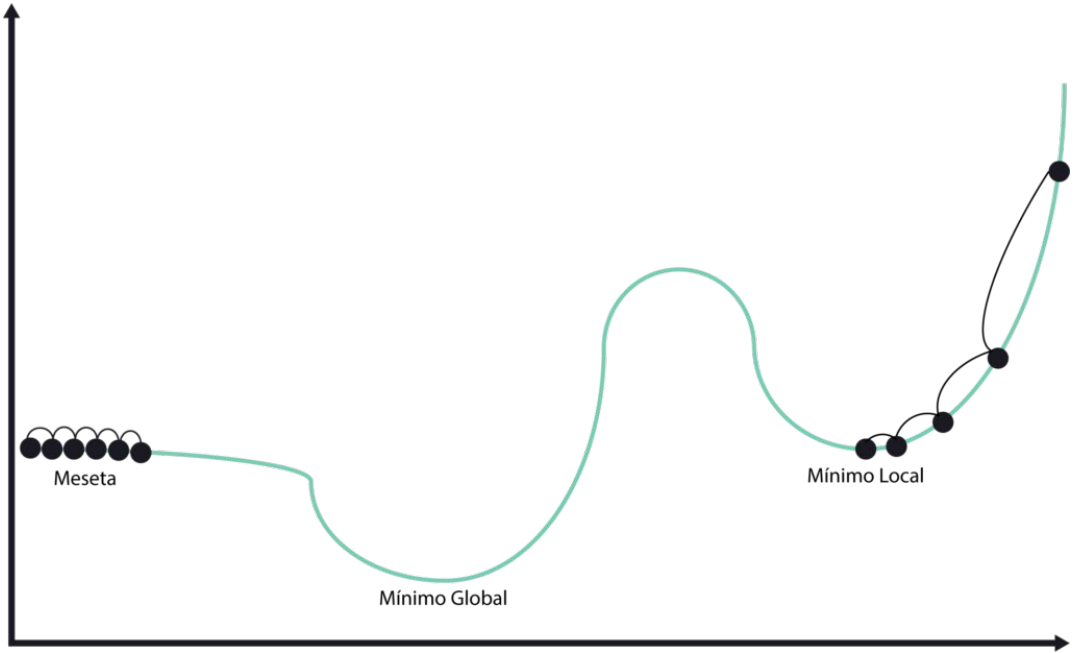
$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^{(t)T} \mathbf{x}_i - y_i) \mathbf{x}_i$$

- Retornar el último $\mathbf{w}$

Cuando se trabaja con conjuntos de datos muy grandes, las operaciones necesarias para calcular la solución cerrada pueden ser computacionalmente prohibitivas debido al costo de calcular y almacenar la matriz XX^T

Descenso de gradiente

- ¿ La solución es única?
- ¿ Depende del punto de inicio?



Funciones y conjuntos convexos

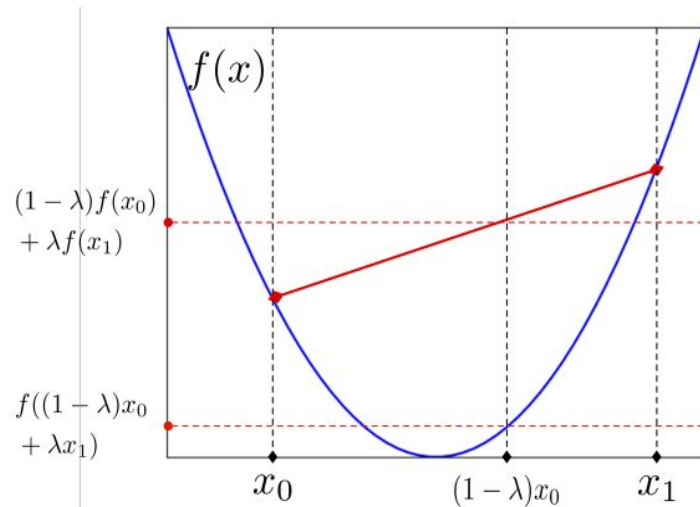
- Una **función** f es **convexa** si para cualquier x_0, x_1 en el dominio de f

$$f((1 - \lambda)x_0 + \lambda x_1) \leq (1 - \lambda)f(x_0) + \lambda f(x_1), \quad 0 \leq \lambda \leq 1$$

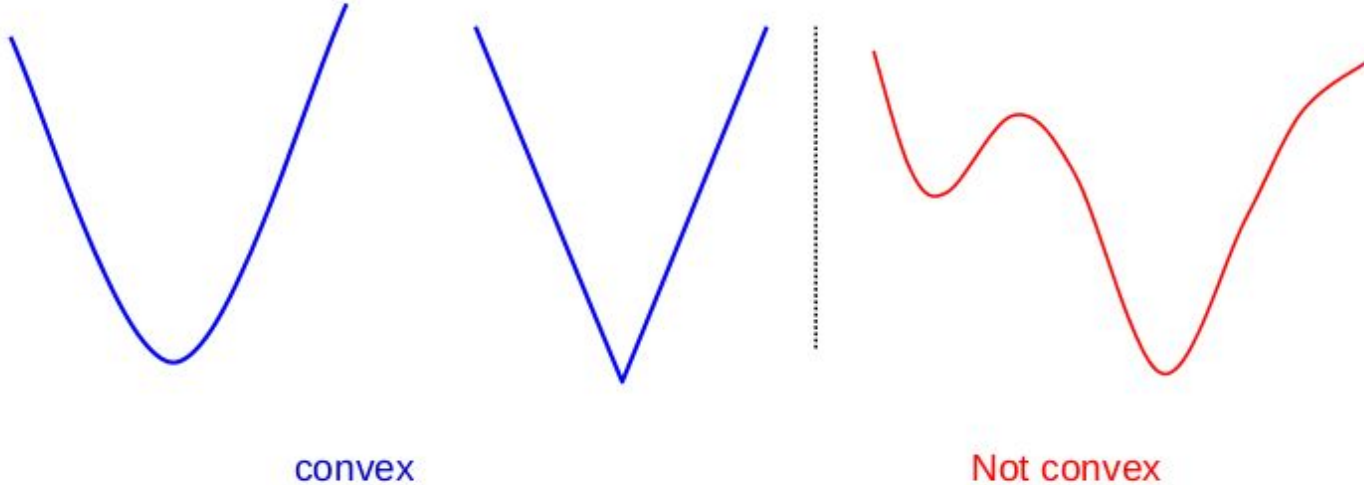
- Un **conjunto** S es **convexo** si para cualquier x_0, x_1 en S

$$(1 - \lambda)x_0 + \lambda x_1 \in S$$

- Intuitivamente la función tiene forma de “cuenco”



Ejemplo de funciones convexas



- La suma no negativa de funciones convexas es convexa

Ejemplo de funciones convexas



SVM

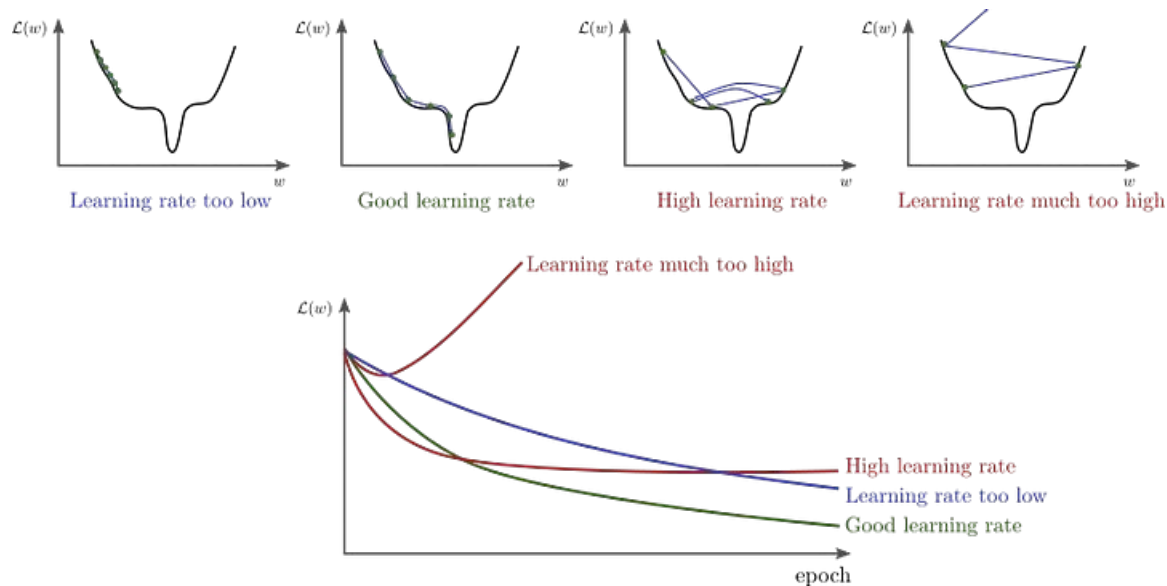
$$\min_{\mathbf{w} \in \mathbb{R}^d} C \sum_i^N \max(0, 1 - y_i f(\mathbf{x}_i)) + \|\mathbf{w}\|^2 \quad \text{convex}$$

¿ Por qué es importante?

- Los puntos críticos (derivada=0) son todos mínimos
- Descenso de gradiente encuentra la solución óptima

Elección de la tasa de aprendizaje

- La tasa de aprendizaje determina la velocidad de convergencia
- Su valor no deberá ser ni muy alto (posibles oscilaciones) ni muy bajo (lentitud, más oscilaciones)



- Si será fija o variable con el tiempo, para evitar oscilaciones cuando estemos cerca del mínimo. Muchas estrategias de adaptación (momentum, scheduling, etc)
- El hiperparámetro más importante junto con el número de iteraciones

El trabajo en aprendizaje supervisado

1. Obtener un conjunto de pares de entrenamiento $D = \{(x_i, y_i)\}, i = 1, \dots, N$
2. Elegir una parametrización para el modelo predictivo $f_{\mathbf{w}}(x)$
3. Elegir una función de costo $l(y, f_w(x))$
4. Resolver el problema de aprendizaje

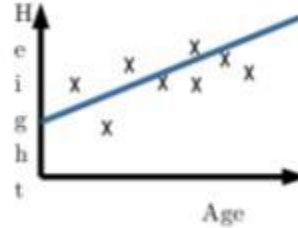
$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N l(y_i, f_{\mathbf{w}}(\mathbf{x}_i)) + \lambda R(w)$$

5. Evaluar, ajustar parámetros, $(\lambda, \eta, T_{\max}, \text{etc})$ volver a iterar...

Definición del modelo predictivo (parametrización)

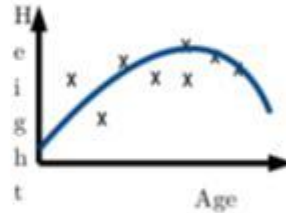
Lineal

$$h_w(x) = \sum_{i=0}^d w_i x_i$$

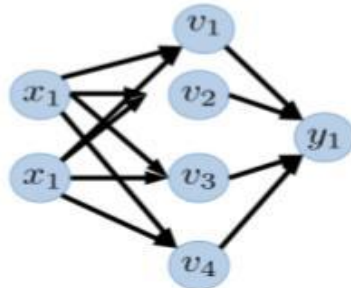


Polinomial:

$$h_w(x) = \sum_{i,j=0}^d w_{ij} x_i x_j$$



Red Neuronal:



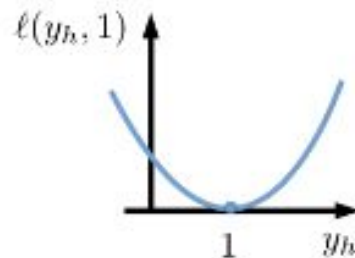
exe :

$$v_1 = \text{sign}(w_{11}x_1 + w_{12}x_2)$$

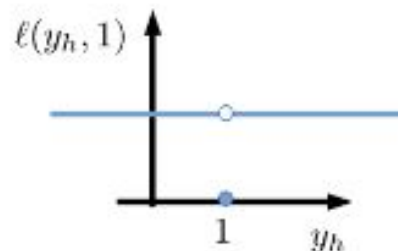
$$v_4 = 1 / (1 + \exp(w_{41}x_1 + w_{42}x_2))$$

Definición de la función de costo

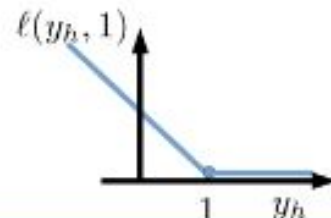
Quadratic Loss $\ell(y_h, y) = (y_h - y)^2$



Binary Loss $\ell(y_h, y) = \begin{cases} 0 & \text{if } y_h = y \\ 1 & \text{if } y_h \neq y \end{cases}$



Hinge Loss $\ell(y_h, y) = \max\{0, 1 - y_h y\}$



Ejemplo: regresión lineal

Linear hypothesis

$$h_w(x) = \langle w, x \rangle$$



L2 regularizer

$$R(w) = ||w||_2^2$$

L2 loss

$$\ell(y_h, y) = (y_h - y)^2$$



Ridge Regression

$$\min_{w \in \mathbf{R}^d} \frac{1}{n} \sum_{i=1}^n (y^i - \langle w, x^i \rangle)^2 + \lambda ||w||_2^2$$

Ejemplo:regresión lineal

Linear hypothesis

$$h_w(x) = \langle w, x \rangle$$



L2 regularizer

$$R(w) = ||w||_2^2$$

Logistic loss

$$\ell(y_h, y) = \ln(1 + e^{-yy_h})$$



Label encoding: $y \in \{-1, +1\}$

$$\mathbb{P}(y = 1|z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\mathbb{P}(y = 0|z) = 1 - \sigma(z) = \frac{1}{1 + e^z}$$

Logistic Regression

$$\min_{w \in \mathbf{R}^d} \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-y^i \langle w, x^i \rangle}) + \lambda ||w||_2^2$$